

MILESTONE 1

Kelompok: ribkacarry

Nomor Kelompok: G-12

List Anggota Kelompok:

1. Ribka Kaylena Sanjaya - 13525045
2. Three Gie Gendhis Sekar Ayoe Jatmiko - 13525144
3. Renata Puspanegara Ninagan - 13525041
4. Muhammad Dhiya Rafi - 13525035

PROGRESS

1. COMPLETED TASKS

No	Task	Penanggung Jawab (NIM)
1.	Fakta dan rule	13525144
2.	Discard Pile	13525144
3.	Start game	13525045
4.	Inisiasi pemain dan validasi nama unik dan jumlah pemain	13525144
5.	mainkanKartu	13525041
6.	ambilKartu	13525041
7.	lihatCommand	13525045
8.	lihatKartu	13525045
9.	cekInfo	13525035
10.	Main (untuk pembuka game saja)	13525144

2. ONGOING TASKS

No	Task	Penanggung Jawab (NIM)
1.	Pembagian Tugas untuk Milestone 2	13525045
2.	Revisi Milestone 1 (masing" anggota)	13525041

3. UNSTARTED TASKS

No	Task	Penanggung Jawab (NIM)
	-	
	-	

RANCANGAN FAKTA

No	Fakta	Deskripsi
1.	<pre>/* FAKTA */ kartu(merah, 0). kartu(kuning, 0). kartu(hijau, 0). kartu(biru, 0). kartu(merah, 1). kartu(kuning, 1). kartu(hijau, 1). kartu(biru, 1). kartu(merah, 2). kartu(kuning, 2). kartu(hijau, 2). kartu(biru, 2). kartu(merah, 3). kartu(kuning, 3). kartu(hijau, 3).</pre>	Mendefinsikan jenis dan warna kartu yang terdapat dalam program, yakni biru, merah, kuning, dan hijau serta angka, reverse, skip, draw, wild, dan mimic.

```
kartu(biru, 3).
```

```
kartu(merah, 4).
```

```
kartu(kuning, 4).
```

```
kartu(hijau, 4).
```

```
kartu(biru, 4).
```

```
kartu(merah, 5).
```

```
kartu(kuning, 5).
```

```
kartu(hijau, 5).
```

```
kartu(biru, 5).
```

```
kartu(merah, 6).
```

```
kartu(kuning, 6).
```

```
kartu(hijau, 6).
```

```
kartu(biru, 6).
```

```
kartu(merah, 7).
```

```
kartu(kuning, 7).
```

```
kartu(hijau, 7).
```

```
kartu(biru, 7).
```

```
kartu(merah, 8).
```

```
kartu(kuning, 8).
```

```
kartu(hijau, 8).
```

```
kartu(biru, 8).
```

```
kartu(merah, 9).
```

```
kartu(kuning, 9).
```

```
kartu(hijau, 9).
```

```
kartu(biru, 9).
```

```
kartu(merah, skip).
```

```
kartu(kuning, skip).
```

```
kartu(hijau, skip).
```

```
kartu(biru, skip).
```

```
kartu(merah, rev). /* REV ->  
REVERSE */
```

```
kartu(kuning, rev).
```

```
kartu(hijau, rev).
```

	<pre> kartu(biru, rev). kartu(merah, drawtwo). kartu(kuning, drawtwo). kartu(hijau, drawtwo). kartu(biru, drawtwo). kartu(hitam, wild). kartu(hitam, drawfour). kartu(hitam, mimic). </pre>	

RANCANGAN RULE

No	Rule	Deskripsi
1.	<pre> :- include('factsRules.pl'). :- include('mekanismeDasar.pl'). :- dynamic(gameStatus/3). :- dynamic(isStart/1). adaDiDeck(H, [H _]). adaDiDeck(H, [_ T]) :- adaDiDeck(H, T). deckLengkap(Deck) :- ambilDeck([], Deck). ambilDeck(A, Deck) :- kartu(Warna, Jenis), \+ (adaDiDeck(kartu(Warna, Jenis), A)),!, ambilDeck([kartu(Warna, Jenis) A], Deck). ambilDeck(A, A). shuffleKartu([], []). shuffleKartu(Deck, [H T]) :- length(Deck, Len), random(0, Len, Idx), </pre>	Perintah startGame

```

nth0 (Idx, Deck, H),
select (H, Deck, Sisa),
shuffleKartu (Sisa, T).

awalBagiKartu (0, Pile, [], Pile) :-
!.
awalBagiKartu (N, [H|T], [H|Kartu],
Sisa) :-
    N > 0,
    N1 is N - 1,
    awalBagiKartu (N1, T, Kartu,
Sisa).

bagikanKartu ([], DrawPile, [],
DrawPile).
bagikanKartu ([player (Nama, Status,
[])|T], DrawPile,
            [player (Nama, Status,
KartuDiTangan)|ListBaru],
DrawPileBaru) :-
    write ('Membagikan ke '),
write (Nama), nl,
    awalBagiKartu (7, DrawPile,
KartuDiTangan, DrawPileSisa),
    write ('kartuDiTangan '),
write (Nama), write (': '),
write (KartuDiTangan), nl,
    bagikanKartu (T, DrawPileSisa,
ListBaru, DrawPileBaru).

verifikasiJumlahPemain (X, 1) :- X >=
2, X <= 4.
verifikasiJumlahPemain (X, 0) :- X <
2.
verifikasiJumlahPemain (X, 0) :- X >
4.

jumlahPemain (X) :-
    write ('Masukkan jumlah pemain
(2-4) : '),
    read (Jumlah),

```

```

    verifikasiJumlahPemain(Jumlah,
Y),
    Y ::= 1,
    X is Jumlah, !.

jumlahPemain(X) :-
    write('Jumlah pemain tidak
valid. Silakan input ulang. '), nl,
    jumlahPemain(X).

isUniquePemain(_, [], 1).
isUniquePemain>Nama,
[player>Nama,_,_)|_], 0) :- !.
isUniquePemain>Nama,
[player(H,_,_)|T], X) :-
    Nama \== H,
    isUniquePemain>Nama, T, X).

loopInput>Nama(0, L, L) :- !.
loopInput>Nama(N, L, PlayerFinal) :-
    N > 0,
    write('Masukkan nama pemain: '),
    read>Nama),
    isUniquePemain>Nama, L, X),
    ( X == 1 -> N1 is N - 1,
append(L, [player>Nama, main, []]),
L1), loopInput>Nama(N1, L1,
PlayerFinal);

    write('Nama sudah digunakan!
Silakan input ulang. '), nl,
loopInput>Nama(N, L, PlayerFinal)).

inisialisasiGame :-
    \+ (isStart(true)),!,
    jumlahPemain(N),
    loopInput>Nama(N, [],
ListPlayer),
    deckLengkap(DeckAwal),
    shuffleKartu(DeckAwal,
DrawPileAwal),
    bagikanKartu(ListPlayer,

```

```

DrawPileAwal, ListTerisi,
DrawPileSisa),
    DrawPileSisa =
[KartuPertama|DrawPileFinal],
    DiscardPile = [KartuPertama],
    retractall(gameStatus(_, _, _)),
    asserta(gameStatus(ListTerisi,
DiscardPile, DrawPileFinal)),
    write('Game siap!'), nl,
    ( ListTerisi =
[player(GiliranNow,_,_)|_] -> true ;
GiliranNow = 'nulllllll' ),
    asserta(isStart(true)),
    tampilStatus.

cekStatus(player(_, menang, []),
menang) :- !.
cekStatus(player(_, kalah, _),
kalah) :- !.
cekStatus(player(_, main, []),
menang).
cekStatus(player(_, main, _),
main).

updateStatusList([], []).
updateStatusList([H|T],
[player>Nama, StatusBaru, Deck)|T2])
:-
    H = player>Nama, _, Deck),
    cekStatus(H, StatusBaru),
    updateStatusList(T, T2).

listPemainAktif([], []).
listPemainAktif([H|T], [H|L]) :-
    H = player(_, main, _), !,
    listPemainAktif(T, L).
listPemainAktif([_|T], L) :-
    listPemainAktif(T, L).

listPemenang([], []).
listPemenang([H|T], [H|L]) :-

```

```

        H = player(_, menang, _), !,
        listPemenang(T, L).
listPemenang([_|T], L) :-
    listPemenang(T, L).

updateGame(ListPlayer, DiscardPile,
DrawPile) :-
    retractall(gameStatus(_, _, _)),
    asserta(gameStatus(ListPlayer,
DiscardPile, DrawPile)).

tampilStatus :-
    nl, write('=== STATUS TERKINI
==='), nl,
    gameStatus(ListPlayer,
DiscardPile, DrawPile),
    ListPlayer = [player>Nama,
StatusBaru, Deck)|SisaPemain],
    write('Turn sekarang      :
'), write>Nama), nl,
    DiscardPile = [Teratas|_Sisa],
    write('Kartu teratas discard :
'), write(Teratas), nl,
    length(DrawPile, JmlDraw),
    write('Sisa DrawPile      :
'), write(JmlDraw), write(' kartu'),
nl,
    write('Pemain
:'), nl,
    tampilListPlayer(ListPlayer).

tampilListPlayer([]).
tampilListPlayer([player>Nama,
Status, Deck)|T]) :-
    write('- '), write>Nama),
    length(Deck, JmlKartu),nl,
    write('Status          :
'), write(Status),nl,
    write('Sisa Kartu di Tangan :
'), write(Deck), nl,
    tampilListPlayer(T).

```


2.	<pre>% menampilkan kartu pemain lihatKartu(ListKartu) :- tampilkanKartu(ListKartu, 1). tampilkanKartu([], _). tampilkanKartu([Kartu Rest], No) :- write(No), write('. '), write(Warna), write('-'), write(Jenis), nl, Noke is No + 1, tampilkanKartu(Rest, Noke).</pre>	Perintah lihatKartu Notes: masih bingung dengan 'disembunyikan' (bagian bonus) mengenai ketentuannya.
3.	<pre>% menampilkan command yang bisa dipilih pemain /* belum pakai state definisi conditional state pemain, (pasti) akan direvisi/ditambahkan lagi setelah diskusi lebih lanjut */ lihatCommand :- nl, write('Aksi utama yang tersedia:'), nl, (bisaMainKartu -> write('1. mainKartu'),</pre>	Perintah lihatCommand

	<pre> nl ; true), (bisaAmbilKartu -> write('2. ambilKartu'), nl ; true), (bisaTantang -> write('3. tantang'), nl ; true), nl, write('Aksi pendukung yang tersedia'), nl, write('1. lihatCommand'), nl, write('2. lihatKartu'), nl, write('3. cekInfo'), nl. </pre>	
	<pre> randomizeIdx(I,N,X) :- random(I, N, X). getCard([H _],1,H). getCard([_ T],I,H) :- I > 1, I1 is I-1, getCard(T, I1, H). removeCard([_ T], 1, T). removeCard([H T], I, [H UT]) :- I > 1, I1 is I-1, </pre>	Mekanisme dasar seperti mainkanKartu dan ambilKartu.

```

        removeCard(T, I1, UT).

pindahGiliran([Pemain|SisaPemain], ListPemainNow) :-
    append(SisaPemain,
[Pemain], ListPemainNow).

adaKartu(Deck, kartu(Warna,
Jenis)) :-
    member(kartu(W, J), Deck),
    (W == Warna ; J == Jenis ;
W == hitam),!.

drawKartu(0, DrawPile, Deck,
DrawPile, Deck) :- !.
drawKartu(N,
[KartuAtas|SisaDraw], Deck,
DrawPileSisa, DeckSisa) :-
    N > 0,
    append(Deck, [KartuAtas],
DeckNow),
    N1 is N - 1,
    drawKartu(N1, SisaDraw,
DeckNow, DrawPileSisa,
DeckSisa).

akhiriGiliran>Nama, Status,
DeckNow, SisaPemain, Discard,
DrawPileNow) :-
    PemainNow = player>Nama,
Status, DeckNow),
    append(SisaPemain,
[PemainNow], ListPemainNow),
    retractall(gameStatus(_, _,
_)),

asserta(gameStatus(ListPemainNow,
Discard, DrawPileNow)),
    (StatusNow == menang ->

```

```

format('~w Menang!~n', [Nama])
; true),
    tampilStatus.

ambilKartu :-
    gameStatus([player(Nama,
Status, Deck)|SisaPemain],
Discard, DrawPile),!,
    drawKartu(1, DrawPile,
Deck, DrawPileNow, DeckNow),
    format('~w mengambil 1
kartu dari deck dan mengakhiri
giliran.~n', [Nama]), nl,
    akhiriGiliran(Nama, Status,
DeckNow, SisaPemain, Discard,
DrawPileNow).

mainkanKartu(N) :-
    gameStatus([player(Nama,
Status, Deck)|SisaPemain],
[KartuTerakhir|SisaDiscard],
DrawPile),

    length(Deck, L),
    (N >= 1, N =< L -> true ;
format('Tidak ada kartu ke ~w!
Pilih kartu antara 1 - ~w.~n',
[N, L]), fail),

    getCard(Deck, N, Played),
    Played = kartu(WarnaPilih,
JenisPilih),
    KartuTerakhir =
kartu(WarnaTerakhir,
JenisTerakhir),
    ((WarnaPilih ==
WarnaTerakhir ; JenisPilih ==
JenisTerakhir ; WarnaPilih ==
hitam) ->

```

```

        kartuCocok>Nama, Status,
Deck, N, Played, SisaPemain,
KartuTerakhir, SisaDiscard,
DrawPile);
kartuTidakCocok>Nama, Deck,
KartuTerakhir)).

kartuCocok>Nama, Status, Deck,
N, Played, SisaPemain,
KartuTerakhir, SisaDiscard,
DrawPile) :-
    Played = kartu(WarnaPilih,
JenisPilih),

    removeCard(Deck, N,
DeckNow),
    (DeckNow == [] -> StatusNow
= menang ; StatusNow = Status),

    format('~w mengeluarkan
kartu: ~w ~w~n', [Nama,
WarnaPilih, JenisPilih]),
    (JenisPilih == drawtwo ->
SisaPemain = [player>NamaNext,
StatusNext,
DeckNext)|SisaLain],
    drawKartu(2, DrawPile,
DeckNext, DrawPileNow,
DeckNextNow),
    PemainNext =
player>NamaNext, StatusNext,
DeckNextNow),
    append(SisaLain,
[PemainNext], SisaPemainEfek),
    format('~w mengambil 2
kartu dari draw pile akibat
drawtwo card!~n', [NamaNext]);
    JenisPilih == rev ->
reverse(SisaPemain,

```

	<pre> SisaPemainEfek), DrawPileNow = DrawPile, write('Urutan giliran dibalik!'), nl; SisaPemainEfek = SisaPemain, DrawPileNow = DrawPile), akhiriGiliran>Nama, Status, DeckNow, SisaPemain, Discard, DrawPileNow). kartuTidakCocok>Nama, Deck, KartuTerakhir) :- write('Kartu tidak cocok! Pilih kartu lain. '), nl, (\+ adaKartu(Deck, KartuTerakhir) -> format('~w tidak punya kartu yang cocok, otomatis mengambil kartu.~n', >Nama]),ambilKartu; true),fail. </pre>	
4.	<pre> :- include('startGame.pl '). startgame :- nl, write('===== =='), nl, write(' Selamat datang di Permainan UNI!'), nl, write('===== =='), nl, nl, write('Aturan :'), nl, write(' - Cocokkan warna atau angka kartu teratas'), nl, write(' - Ketik DrawPile untuk </pre>	Berisi perintah pembuka untuk game

	<pre> ambil kartu dari DrawPile'), nl, write(' - Pemain yang kartunya habis maka MENANG'), nl, write(' - Jika DrawPile habis dan tidak bisa main, KALAH'), nl, nl, inisialisasiGame, nl, write('--- Mulai bermain ---'), nl. </pre>	
5.	<pre> :-include(startGame.pl). /* Basis untuk tulisPemain */ tulisPemain(Urutan, JumlahPemain,, []) :- Urutan > JumlahPemain, !. /* Rekurens tulisPemain */ tulisPemain(Urutan, JumlahPemain, [Nama SisaNama]) :- write('Nama pemain'), write(Urutan), write(': '), write(Nama), nl, player(Nama, DaftarKartu), length(DaftarKartu, X), write('Jumlah kartu : '), write(X), nl, nl, NextUrutan is Urutan + 1, tulisPemain(NextUrutan, JumlahPemain, SisaNama). /* Basis untuk tulisListUrutan */ tulisListUrutan([NamaTerakhir]) :- write(NamaTerakhir), write('.'), nl, !. /* Rekurens tulisListUrutan */ tulisListUrutan([Nama Sisa]) :- </pre>	Perintah cekInfo Mekanisme dasar untuk melihat state permainan saat itu

```
        write>Nama), write(' - '),
        tulisListUrutan(Sisa).

cekInfo :-
    game_status(ListPlayer,
DiscardFile, DrawFile),
    write('Kartu discard top: '),
write(DiscardFile), nl, nl,
    write('Urutan pemain: '),
    tampilListPlayer(ListPlayer),
nl,
    tulisPemain(1, JumlahPemain,
ListUrutan).
```